In both these scenarios, the service layer can be considered as a separate API, with specific business goals and requirements. These requirements will be driven by the client applications that consume the services, and which may be developed by different teams. In BDD terms, the client application and the service API should be considered separately, each with its own set of scenarios and executable requirements.

A service-based architecture works well with BDD. By definition, services are designed to be reusable, so it makes sense to define them clearly and cleanly, and to document them with worked examples. When you design a new service using BDD, you begin by describing the requirements in the form of practical examples of how the service will be used. This tends to produce cleaner, more focused services that are easier to understand. If it's hard to find clean scenarios with well-defined input parameters and expected outcomes, then the service being designed may need a little more thought.

This approach also makes the reuse of these services easier. Well-written BDD scenarios give a clear description of what a service is meant to do and how it works, and the automation steps give worked examples of how to interact with a service that new developers can use to get started quickly.

One of the most popular ways to implement an SOA architecture is to use web services. REST web services in particular are becoming increasingly widespread, though the more heavyweight SOAP web services are still widely used.

Let's look at a few examples of working with both REST and SOAP in Java and .NET.

RESTful web services with Cucumber and Java

Imagine you're designing web services for the Flying High Frequent Flyers application. One of the core services might be to display the details of a given flight. This service is required in several different screens and by the back-end payment-processing system:

```
Feature: Retrieve information about a given flight
  Scenario: Find flight details by flight number
    Given I need to know the details of flight number FH-101
    When I request the details about this flight
    Then I should receive the following:
    | flightNumber | departure | destination  | time   |
    | FH-101       | MEL       | SYD          | 06:00  |
```

For simplicity, this example supposes that flight FH-101 is a well-known entity in the system. The key value in this scenario is discovering precisely what information you need to retrieve about a given flight.

Note that although you're implementing a web service, this scenario is implementation-neutral and would be perfectly understandable by team members not fluent in JSON or XML. It's also a good first approach to the problem that will help the team agree on the goals of a particular web service, its input parameters, and what information the service will provide.

You could automate this scenario using Cucumber in Java as shown here: